



Applied Statistic and
Machine Learning
CA one Assignment

Submitted by.
Rahul Pagar

Submitted on
15/11/2024

Report:

Supervised Machine Learning- Classification

Question 1. Data Preparation (What steps would you take to prepare your data)?

Answer:

Steps involved in data preparation are as follows:

1.Data Import:

- Imported few packages that will help to build the model packages such as, pandas, Sklearn pre_processing, Sklearn model_selection, imlearn.
- My data file is in csv format, so I used **read_csv** to import file into python/colab environment.
- We used **.head()** function to verify the imported file is in readable format. And to ensure the same file on which we will be operating. To see the rows output we can see the any number in parenthesis with head function.

2.Data understanding:

- This step is performed to understand the dataset structure datatypes, statistical summary, rows and columns information.
- I used **.shape** function to get the information about 17 columns and 4521 rows.
- Then we used **.describe()** function is used to get the statistical summary of dataset.
- The **.info()** can be used to understand the any null values in dataset also it gives information about column names and their data types, the number of non-null values.

3.Data cleaning:

- This step is performed address the missing values in dataset.
- No missing values in dataset as we get this information in data understanding step with the help of **.info()** function.

4.Data encoding:

- This step is performed to convert the categorical variables into numerical variables.
- There are 07 numerical and 10 categorical variables in a dataset.

- Following columns stores categorical values all columns must convert to numerical as ML algorithm can't handle categorical data. Column names: Job, marital, education, default, housing, loan, contact, month, outcome, y
- The **.unique()** function is used to extract the unique values from columns.
- I used **map** function to convert the 04 categorical variables (default, housing, loan, y) into binary codes that have only two types of data. yes and no. In this assignment yes is mapped with 1 and no mapped with 0.
- One hot encoding: I have generated dummy columns for remaining 10 categorical variables (job, marital, education, contact, month, outcome) that have more than two types of data. The columns with multiple category data transformed from categorical data to numerical data by using **get_dummies()** function.
- Now we have 49 columns and 4521 rows.

5. Selecting dependent and independent variable:

- The dependent variable is those that the model is trying to predict.
- Independent variable is variable that is used to predict the dependent variable.
- In bank dataset y (has the client subscribed a term deposit) is dependent variable.
- In bank dataset except y column all remaining columns are independent variable.
- New variables created and dependent and independent variables are allotted to these new variables.

6. Data scaling / Standardisation:

- This step is performed to normalize the range of values in dataset. This is done by transforming the data into consistent scale. Scaling helps ensure that each feature contributes equally to the analysis process.
- Data scaling is performed on independent variable (x)
- The **StandardScaler().fit_transform()** is used to scaled the independent variable.

7. Splitting the dataset in train and test:

- This step is performed to split the train and test to evaluate the model.
- For splitting the scaled independent variable following function is used:
Train-Test Split: **train_test_split(x, y, test_size=0.3, random_state=42)**
- For evaluating the model scaled dataset split in 70:30. I have allocated 70% data for training it is sufficient data to learn the patterns and trends. Allocating 30% data for testing so that I can check the how well my model work with unseen data or with new examples.

8.Data balancing:

- This step is performed to ensure that all the data points in bank dataset have equal importance when it is used for statistical analysis.
- In this dataset training set(y_train) is imbalance and it is required to well-balanced training set ensures balanced learning.
- Imbalanced datasets can be balanced by down sampling the majority and up sampling the minority. There is, however, a simpler way to oversample: designing new synthetic samples that are close to current minority class samples. SMOTE is used for this (Synthetic Minority Oversampling Technique)
- y_train has the imbalance data in 1 and 0. 0 has the more datapoints than 1. So, it is important to balance this otherwise this can lead to biased models.
- I used SMOTE technique as it makes data more even. I have used SMOTE as this technique instead of just copying existing examples in the minority class SMOTE creates new significant examples (synthetic data) based on existing data.
- This function is used for data balancing: **smote.fit_resample()**

Question 2. Model Hyperparameter Tuning (Which hyperparameters would you tune and why? How would you tune them?)

Answer:

In this example we build two models.

1. Random Forest classification
2. Support vector machines classification

- Hyperparameter tuning is the process of selecting the best set of hyperparameters for a machine learning model to get a good performance.
- Hyperparameter tuning is important to enhance model accuracy. By hyperparameter tuning we can achieve robust machine learning model and avoid the overfitting as well as underfitting.

1. Random forest: There are 04 parameters in the random forest (max_features, n_estimators, random_state and criterion) but out of 04 following two hyperparameter can be tuned in this example.

a) Maximum features:

- This parameter allows to control the number of features to be considered when looking for best split
Sqrt, log2 and None option available for tuning. Tuning can be done by checking sqrt and log2 to see which provides better performance.

With sqrt option during each split, it will take square root of the total number of features. It also gives some flexibility to decision tree to capture different patterns and not let the decision tree to overfit. With log2 option during each split it will take base 2 logarithmic of the total number of features.

➤ Tuning steps:

I gave the range for max_features as ['sqrt', 'log2', None]

Use GridSearchCV function to tune the of max_features. (This function tries out each combination of the specified max_features values and fits the model). This functions also uses cross validation which means it splits the data into five equal sized groups (when cv=5) and uses 4 to train and 1 to test the result. This process repeats 5 times, with each part getting a chance to be the test data once. I have taken cv=5 for cross fold validation.

Decide the performance metric for example precision, accuracy, F1 score based on requirement. In this assignment precision is used.

Fit the grid search classifier on training set.

Use **grid_search.best_params** function and **best_params** give the best value during the search to get the best parameter/optimal parameter.

Then refit the random forest with optimal n_estimator.

Evaluate the refitted random forest with optimal **max_features** parameter.

Then check the performance of random forest in terms of accuracy, precision and recall.

b) **n_estimators:**

- This parameter controls the number of decision trees in the forest, and increasing it can give better ensemble performance.

More trees can improve model performance by providing better averaging and reducing variance.

➤ Tuning steps:

Select the range for n_estimator. Here I used list to give the range for n_estimator. I have given this range [50,100,150,180,210,235,280,320,355]

Use GridSearchCV function to tune the of n_estimator. (This function tries out each combination of the specified n_estimator values and fits the model).

This functions also uses cross validation which means it splits the data into five equal sized groups (when cv=5) and uses 4 to train and 1 to test the result. This process repeats 5 times, with each part getting a chance to be the test data once. I have given the cv=5.

Decide the performance metric for example precision, accuracy, F1 score based on requirement. In this assignment precision is used.

Fit the grid search classifier on training set.

Use **grid_search.best_params** function and **best_params** give the best value during the search to get the best parameter/optimal parameter.

Then refit the random forest with optimal n_estimator.

Evaluate the refitted random forest with optimal **n_estimator** parameter.

Then check the performance of random forest in terms of accuracy, precision and recall.

➤ **Conclusion:**

For bank data set with random forest algorithm **n_estimators** and **max_features** are tuned after hyper parameter tuning, I got optimal n_estimator as 355 and max_features as log2. These two (n_estimator and max_features) parameters tuned to get good performance and robust model.

2. Support vector machines: Following hyperparameters can be tuned in SVM.

a) Kernel:

- The kernel function defines how the SVM separates data points in a transformed feature space. Common kernel includes: Linear (for linearly separate data), Polynomial (for nonlinear relationship), RBF (for more complex and non-linear patterns), sigmoid (map input data in higher dimension). Tuning can be done by checking linear, polynomial and sigmoid to see which provides better performance.

➤ Tuning steps:

Give the options for Kernel it includes (Linear, polynomial, rbf and sigmoid)
Use GridSearchCV function to tune the Kernel parameter (This function tries out each combination of the specified regularization parameter values and fits the model).

This functions also uses cross validation which means it splits the data into five equal sized groups (when cv=5) and uses 4 to train and 1 to test the result. This process repeats 5 times, with each part getting a chance to be the test data once.

Select the scoring parameter e.g. Precision, accuracy, F1 call based on requirement. In this assignment precision is used.

Fit the grid search classifier on training set.

Use **grid_search.best_params** function and **best_params** give the best value during the search (to get the best parameter/optimal parameter).

Then refit the support vector machines with optimal Kernel.

Evaluate the refitted SVM with optimal Kernel.

b) Regularization parameter:

- This parameter is critical in building robust and accurate algorithms to prevent overfitting and underfitting. A higher C value may result in overfitting while the low value of c may result in underfitting. so, it's important to select optimal regularization parameter to increase the performance of the model.

➤ Tuning steps:

Give the range for regularization parameter(C). I have given this range [0.001,0.01,0.1,1,10].

Use GridSearchCV function to tune the of regularization parameter (This function tries out each combination of the specified regularization parameter values and fits the model).

This functions also uses cross validation which means it splits the data into five equal sized groups (when CV=5) and uses 4 to train and 1 to test the result. Select the value for cross validation cv.

Decide the performance metric e.g. Precision, accuracy, F1 call based on requirement. In this assignment precision is used.

Fit the grid search classifier on training set.

Use **grid_search.best_params** function and **best_params** give the best value during the search (to get the best parameter/optimal parameter).

Then refit the support vector machines with optimal regularization parameter.

Evaluate the refitted SVM with optimal regularization parameter.

Then check the performance of SVM in terms of accuracy, precision, F1 score and recall.

➤ **Conclusion:**

For bank data set with SVM classifier, **Regularization parameter** and **Kernel** is tuned to enhance the performance and avoid the overfitting. After tuning I got 10 as an optimal value for regularization parameter and rbf as an optimal value for Kernel.

3.Choice of Evaluation Metric (Which metric would be suitable for model evaluation and why?)

Answer:

➤ **Confusion matrix of the RF classifier provides the following details:**

TP: True Positives: Customers subscribed correctly identified by the model = 49

FP: False Positives: Customers predicted to subscribe in actual not subscribed = 40

TN: True Negatives: Customers not subscribed and correctly identified = 1165

FN: False Negatives: Customers subscribed but missed by the model = 103

➤ **Confusion matrix of the SVM classifier provides the following details:**

TP: True Positives - Customers subscribed correctly identified by the model = 67

FP: False Positives- Model predicted to subscribe but in actual not subscribed = 86

TN: True Negatives- Customers not subscribed and correctly identified = 1119

FN: False Negatives- Customers subscribed but missed by the model = 85

- This matrix shows number of negative cases are more as compared to positive cases. In our assignments data set has more no cases than yes cases for deposit subscription.
- Accuracy indicates the proportion of total correct predictions to the total predictions. It also suggests that how effectively model classify the results.
- Recall reflects the model's ability to identify all actual "yes" cases, minimizing false negatives e.g., missing potential subscribers. Recall indicates the model's ability to identify the potential customers which are probably subscribe the deposit. Recall also suggest that how well model identified the actual subscribers from total subscribers.
- Precision measures the proportion of positive predictions that are correct. Precision also indicates the proportion of predicted "yes" cases that are correct, helping avoid false positives. This is important in banking as based on this we can avoid the aiming of those customers who are not likely to subscribe.
- F1 is harmonic mean of precision and recall. It is also suitable for model evaluation as it balances both precision and recall and provides a single value to measure the effectiveness of the model.
- **Conclusion:**
 I got good accuracy with random forest and SVM classifier for this assignment I got 89.46% accuracy with Random Forest and 87.39% accuracy with SVM. To summarize Accuracy indicates the correctness of the model, Precision and recall are also suitable with accuracy to evaluate the model as it measures how effectively bank target the potential customers.
 So, **Accuracy, Recall and Precision** are suitable for evaluating the random forest and support vector machine classifier.

4. Overfitting avoidance mechanism (Which mechanism (feature Selection/ regularization) would you use and why?)

Answer:

- Overfitting occurs when model learn well on a training data. The model performs extremely well on training data but poorly on unseen data.
- In this assignment, **no overfitting** was observed in either the random forest or support vector machine models, based on their accuracy values
- However, if overfitting occurs it can be avoided or reduced by the following methods in the classification task:

1. Random forest classifier:

a) Regularization:

- In this method overfitting is controlled by tuning the parameters.
- We can tune the following parameters `max_features` and `n_estimators`.
- `Max_features`: This parameter allows to control the number of features to be considered when looking for best split.
- `n_estimators`: This parameter controls the number of decision trees in the forest, and increasing it can give better ensemble performance. More trees can improve model performance but it can increase by providing better averaging and reducing variance.

b) Feature selection:

- In this method we select only most relevant features, Random Forest can become less complex with feature selection which helps to reduce the overfitting.
- In feature selection we select the relevant features and remove the redundant and irrelevant features which helps the model to focus on important data and on meaningful patterns.

2. SVM Classifier:

- a) Regularization: In this method we control the overfitting by controlling regularization parameter `C` and kernel. This `C` parameter controls the decision boundary. With high `C` value model try to cover each point which makes the model overfit and low `C` value allows the model to ignore some misclassifications. This results in a smoother decision boundary with a wider margin, reducing the model's tendency to overfit. The kernel function defines how the SVM separates data points in a transformed feature space.

➤ Conclusion:

In this assignment no overfitting observed with both classifiers random forest as well as on SVM.

But if overfitting occurs in **Random Forest** model, I would recommend **feature selection** method to reduce the overfitting. As in random forest while building decision tree it selects the feature at each split.

If overfitting occurs in **SVM** model, I would recommend the **Regularization** method to reduce the overfitting as it controls the decision boundary and hyperplane.

5. Results analysis

- a). Which of the two models (random forest or support vector classifier) would you recommend for deployment in the real-world?
- b). Is any model underfitting? If yes, what could be the possible reasons?

Answer:

- In this assignment the goal is to predict whether client has subscribed a term deposit based on given data.
- I applied both feature selection and regularization (hyperparameter tuning) to the random forest classifier, and observed better performance with regularization method.

following results after hyperparameter tuning:

```
# Refit the random forest on the optimal n_estimators parameter
RF2=ensemble.RandomForestClassifier(n_estimators=355,criterion='entropy',max_features='log2',random_state=42)
RF2.fit(x_train,y_train) # fitting random forest to training data
y_pred=RF2.predict(x_test)

# Evaluating the refitted random forest with the optimal n_estimator parameter
from sklearn import metrics
Accuracy= metrics.accuracy_score(y_test,y_pred) # Calculating the accuracy of the model build
Recall= metrics.recall_score(y_test,y_pred) # Calculating the recall of the model build
Precision= metrics.precision_score(y_test,y_pred) # Calculating the precision of the model build
F1= metrics.f1_score(y_test,y_pred)
print('Accuracy:', Accuracy)
print('Precision:', Precision)
print('Recall:', Recall)
print('F1 Score:',F1)

Accuracy: 0.894620486366986
Precision: 0.550561797752809
Recall: 0.3223684210526316
F1 Score: 0.4066390041493776
```

Accuracy: 0.894620486366986
Precision: 0.550561797752809
Recall: 0.3223684210526316
F1 Score: 0.4066390041493776

- Accuracy: 0.8946 (89.46%)
- Precision: 0.5505 (55.05%)
- Recall: 0.3223 (30.23%)
- F1 score: 0.4066 (40.66%)

- With support vector machine classifier, I got following results after hyperparameter tuning:

```
# Predicting the test values using the classifier
y_pred=svm_classifier_rbf.predict(x_test)

# Evaluating the model based on chosen kernel
from sklearn import metrics

Accuracy= metrics.accuracy_score(y_test,y_pred)
Precision=metrics.precision_score(y_test,y_pred)
Recall = metrics.recall_score(y_test,y_pred)
F1= metrics.f1_score(y_test,y_pred)

print('Accuracy:', Accuracy)
print('Precision:', Precision)
print('Recall:', Recall)
print('F1 score:', F1)

Accuracy: 0.8739867354458364
Precision: 0.43790849673202614
Recall: 0.4407894736842105
F1 score: 0.43934426229508194
```

```
Accuracy: 0.8739867354458364
Precision: 0.43790849673202614
Recall: 0.4407894736842105
F1 score: 0.43934426229508194
```

- Accuracy: 0.8739 (87.39%)
 - Precision: 0.4379 (43.79%)
 - Recall: 0.4407 (44.07%)
 - F1 score: 0.4393 (43.93%)
- In this assignment with both models, I got low precision value (55.05% with RF and 43.79% for SVM) which reflect that how many positive predictions are correct. It also suggests that half of the prediction is false positive cases. With low precision value in real world situation model would target customers incorrectly. Also, I achieved low recall value (44.07% with SVM and 32.23% with RF) which suggest that model not identified positive instances in total number of actual positive class. It means model missed more than half of the

true positive cases. With low recall value in real world situation model will not target potential customers effectively.

- Considering performance metric of both random forest and support vector machines classifier. I would recommend neither Random Forest classifier nor SVM classifier is suitable for deployment in real world because of its low precision and low recall value. Both models would ineffectively target the customers. If we want to deploy this improving recall and precision value is recommended to target the potential customers.
- But If I have to choose between Random Forest and SVM for a real-world deployment based on accuracy value the I would recommend Random Forest classifier. The accuracy of random forest classifier and SVM classifier is good. With random forest I achieved 89.46% accuracy which is better than 87.40% with SVM. Random forest classifier higher accuracy indicates that random forest makes more correct predictions. This mean that model will perform well on unseen or new data.

➤ **Conclusion:**

So, I would recommend Random Forest classifier for real world deployment.

I have selected random forest classifier for real world deployment based on **Accuracy** value. Also, random forest is ensemble method that combines multiple decision trees to improve machine learning models performance and robustness.

b). Is any model underfitting? If yes, what could be the possible reasons?

- Underfitting occurs when model is too simple and failed to capture the patterns in the data.
- I achieved good accuracy 89% with RF classifier and 87% on SVM classifier Both models are performing well, and based on their accuracy values, I can conclude that neither of the models is underfitting.

➤ **Conclusion:**

Both the Random Forest and SVM models are performing well, with no signs of underfitting.

- Underfitting occurs mainly because of following reason:
 1. When model complexity is low.
 2. Model is not trained properly i.e. model is too simple
 3. Poor feature selection
 4. Insufficient data that is dataset is small and doesn't capture the patterns in the data.
 5. Class imbalance (e.g. more no cases than yes).